

SOFTWARE TESTING · STRUCTURAL COVERAGE

Code Coverage

Criteria · MC/DC · tools by language · CI dashboards · mutation testing · AI

A two-hour lecture — from the metric to what it cannot prove.

CS423 · CSC15003 — Software Testing (AI-First) Faculty of Information Technology · HCMUS

Rebuilt from the FIT Code Coverage & Coverage-Tools decks · methods & tools verified 2024–2026.

Roadmap

Six parts, about two hours.

A • Fundamentals

What coverage is, how it's measured, and why coverage $\neq$ quality.

B • Criteria

Statement $\rightarrow$ branch $\rightarrow$ condition $\rightarrow$ MC/DC $\rightarrow$ path; the subsumption hierarchy.

C • Reports & targets

Instrumentation, reading reports, sane targets, diminishing returns & gaming.

D • Tools by language

JaCoCo, Istanbul/c8/Vitest, coverage.py, gcov/llvm-cov, coverlet, Go, Rust.

E • CI & dashboards

Codecov, Coveralls, SonarQube; patch/diff coverage; quality gates.

F • Beyond + AI

Mutation testing, coverage-guided fuzzing, LLM test generation (2024–26).

PART A

Fundamentals

The metric, the mechanism, and the caveat every engineer must internalize.



What is code coverage?

A metric for how much of the source the test suite actually executes.

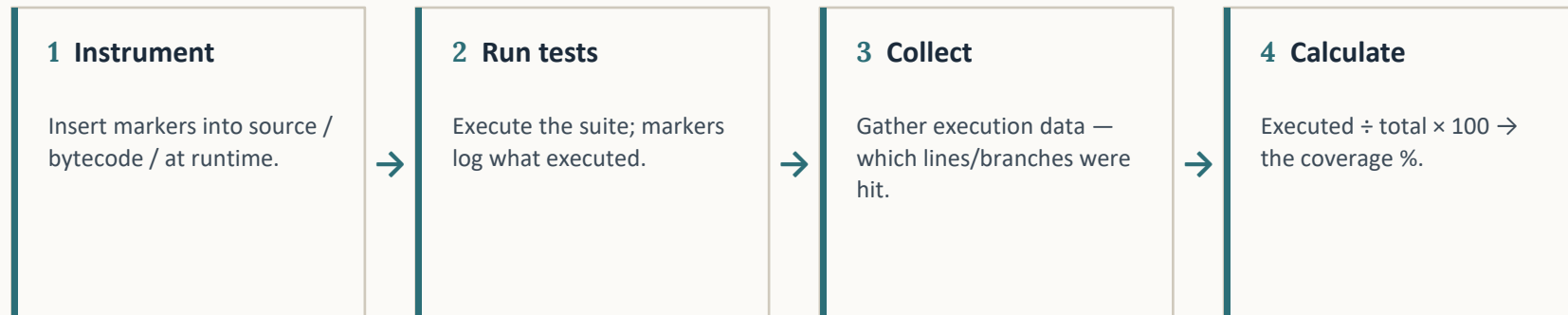
Coverage = the % of code elements (lines, branches, conditions...) executed at least once while the tests run.

1. **Finds untested code** — highlights lines, branches and functions no test ever reaches.
2. **Gauges thoroughness** — a quantitative signal of how much behaviour is exercised.
3. **Exposes dead code** — code never executed by any test surfaces as 0%.
4. **Guides new tests** — points precisely at what to test next.

Figure 1. Coverage answers ‘what ran?’ — never ‘what was checked?’.

How coverage is measured

Four steps — the code is instrumented so it records what runs.



INSTRUMENTATION JaCoCo rewrites Java bytecode via a Java agent; Istanbul transforms JS source; c8 & Go use the runtime's built-in coverage — no source rewrite.

The caveat: coverage $\neq$ good tests

100% coverage proves code ran — not that behaviour was checked.

COVERED — BUT UNTESTED

```
def discount(price, is_member):  
    return price*0.9 if is_member \  
        else price  
  
# test_discount():  
assert discount(100, True) == 90  
assert discount(100, False) == 100  
  
# never tests: negatives, 0,  
# float rounding, huge values
```

Never checked

Negative prices, zero, float rounding, huge values — all “covered”, none verified.

Assertion-free tests

A test that calls code and asserts nothing still yields full coverage and catches zero bugs.

RULE OF THUMB coverage is necessary, not sufficient. Aim high but keep tests meaningful — chase assertions and mutation score (Part F).

PART B

Coverage criteria

From weakest to strongest — each criterion demands more of your tests.

B

Function & statement (line) coverage

The two weakest, most common criteria.

Function coverage

Has each function / method been called at least once? = $\text{functions called} \div \text{total}$.

Statement / line coverage

Has each executable statement run? = $\text{statements executed} \div \text{total}$. The default most tools report.

REFERENCE CODE (used throughout Part B)

```
void foo(int a, int b, int x) {  
    if ((a > 1) && (b == 0)) x = x / a;  
    if ((a == 2) || (x > 1)) x = x + 1; }
```

One test a=2, b=0, x=4 runs both bodies → 100% statements...

...yet it never tries the false branches — weak.

Branch (decision) coverage

Every branch of every decision taken both true and false.

REFERENCE CODE

```
void foo(int a, int b, int x) {  
    if ((a > 1) && (b == 0)) x = x / a;  
    if ((a == 2) || (x > 1)) x = x + 1; }
```

Both outcomes of each if:

{a=2, b=0, x=4} → both decisions TRUE

{a=0, b=0, x=0} → both decisions FALSE

KEY FACT decision coverage SUBSUMES statement coverage — hit every branch and you hit every statement, but not vice-versa.

WHY IT MATTERS branch coverage is the practical sweet spot most teams gate on — it catches the “forgot the else” class of bugs.

Condition & condition/decision coverage

Look inside the decision at each Boolean sub-expression.

Condition coverage

Each atomic Boolean ($a > 1$, $b == 0$, $a == 2$, $x > 1$) takes both true and false. Does NOT necessarily give decision coverage.

Condition/decision coverage

Condition coverage AND decision coverage satisfied at the same time.

SUBTLE TRAP you can flip every condition true/false yet leave a decision's overall outcome unchanged — that's why condition $\neq$ decision coverage.

Example set: $\{a=2, b=0, x=4\}$ and $\{a=1, b=1, x=1\}$ exercise each condition both ways.

MC/DC — Modified Condition/Decision Coverage

Each condition must INDEPENDENTLY change the decision's outcome.

For n conditions, MC/DC needs $\sim n+1$ well-chosen tests (vs 2^n for full multiple-condition): holding others fixed, flipping one condition flips the result.

#	A	B	C	$(A \vee B) \wedge C$
1	T	F	T	T
2	F	F	T	F
3	F	T	T	T
4	T	F	F	F

Reading the table

1 $\leftrightarrow$ 2 : only A flips $\rightarrow$ output flips (A independent)

3 $\leftrightarrow$ 2 : only B flips $\rightarrow$ output flips (B independent)

1 $\leftrightarrow$ 4 : only C flips $\rightarrow$ output flips (C independent)

Multiple-condition & path coverage

The strongest structural criteria — and why they're usually impractical.

Multiple-condition coverage

Every combination of conditions in a decision — all 2^n truth-table rows. For $(A \vee B) \wedge C$ that's 8 tests. Thorough, but explodes.

Path coverage

Every feasible execution path end-to-end. Loops make paths effectively infinite → unattainable in real code.

PRACTICE path coverage is the theoretical ceiling; teams settle for branch (everyday) or MC/DC (safety-critical). Pick the criterion the risk justifies.

The subsumption hierarchy

Satisfy a stronger criterion and you automatically satisfy the weaker ones it contains.

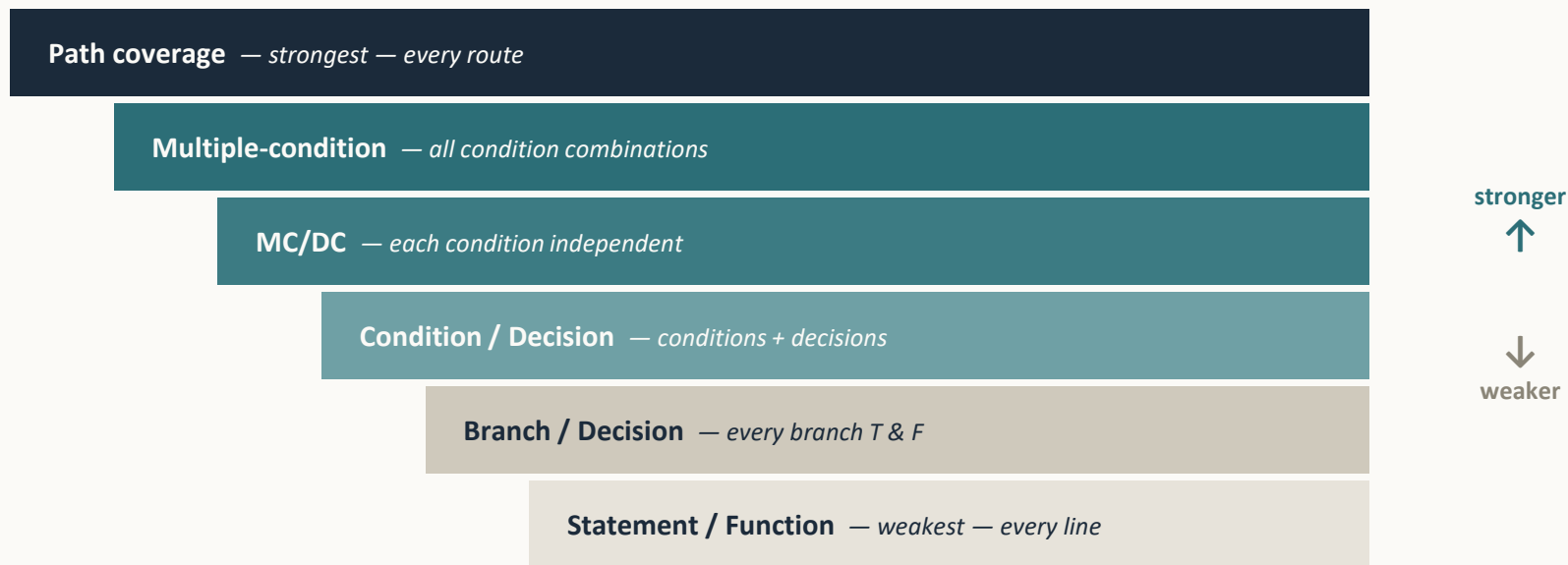


Figure 2. Each level subsumes everything below it.

Worked example — tests per criterion

The classic `foo(a, b, x)`: how many tests each criterion needs.

Criterion	Test set {a, b, x}	Minimum tests	Idea
Statement	{2, 0, 4}	1	run each statement once
Branch / decision	{2,0,4}, {0,0,0}	2	each decision true and false
Condition	{2,0,4}, {1,1,1}	2..n	each Boolean sub-term T/F
MC/DC	n+1 well-chosen	n+1	each condition flips outcome
Path	all feasible paths	2^k	every route through the code

Table 1. Test counts grow sharply toward path coverage.

TAKEAWAY more coverage costs more tests with diminishing returns — match the criterion to the code's risk.

Where MC/DC is mandated

For safety-critical software, branch coverage isn't enough — standards require MC/DC.

DO-178C — avionics

Software whose failure is catastrophic (Level A) must achieve MC/DC. The standard for airborne systems; supported by DO-330/DO-248C.

ISO 26262 — automotive

MC/DC is highly recommended at the software-unit level for ASIL D (the highest automotive safety-integrity level).

WHY MC/DC (NOT MULTIPLE-CONDITION) MC/DC gives near-exhaustive assurance at $\sim n+1$ tests instead of 2^n — feasible for the large decisions in control software.

PART C

Reports, targets & limits

Reading the numbers, setting sane targets, and not gaming the metric.



Reading a coverage report

Each metric answers a different question — read them together.

Metric	What a high value means	But watch for
Statement / line	most lines were executed	untested branches & conditions
Branch / decision	both sides of each if ran	untested compound conditions
Function / method	functions were invoked	invoked, not thoroughly tested
Condition	Boolean sub-terms tried both ways	decision outcome may be unchanged

FORMATS reports come as colour-highlighted HTML (red = missed lines), plus XML/CSV for CI ingestion.

Targets, diminishing returns & gaming

Set a sane bar, spend effort where risk warrants, and don't let the number become the goal.

A sane target

80% on new/changed code is a common gate (e.g. JaCoCo COVEREDRATIO ≥ 0.80). Not a global mandate.

Diminishing returns

the last 10–15% (error handlers, defensive branches) costs disproportionately — spend it where risk warrants.

Gaming the metric

coverage becomes a target → tests that execute code without asserting. Goodhart's law in action.

The fix

gate on PATCH coverage (Part E) and validate with MUTATION score (Part F), not a global %.

PART D

Tools by language

Every mainstream language has a mature, mostly-free coverage tool.



Java — JaCoCo

The de-facto standard: on-the-fly bytecode instrumentation via a Java agent.

MAVEN CONFIG

```
<!-- pom.xml -->
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
</plugin>
mvn test → target/site/jacoco/index.html
```

Run mvn test → HTML report

Ecosystem Maven/Gradle · Jenkins · IntelliJ · EclEmma

Kotlin → Kover ./gradlew koverHtmlReport

Legacy: avoid Cobertura & EMMA are unmaintained

JavaScript / TypeScript

Istanbul is the classic; V8-native coverage (c8, Vitest) is the modern trend.

FOUR WAYS TO GET JS COVERAGE

```
# Istanbul CLI  
nyc mocha  
# V8-native (fast, no source transform)  
npx c8 node test.js  
# Vitest (v8 or istanbul provider)  
vitest run --coverage  
# Jest built-in  
jest --coverage
```

Vitest v3.2+

AST-remaps V8 data to Istanbul-level accuracy
— fast and precise.

Jest report

terminal %Stmts / Branch / Funcs / Lines.

Python — coverage.py & pytest-cov

coverage.py is the engine; pytest-cov is the plugin around it.

PYTEST-COV

```
# statement coverage + HTML report  
pytest --cov=pkg --cov-report=html  
# -> htmlcov/index.html  
# add branch coverage  
pytest --cov=pkg --cov-branch
```

Branch flag `--cov-branch` turns on decision coverage (off by default).

Reports term, html, xml (for CI), json.

C / C++ — gcov, lcov & llvm-cov

Coverage-instrumented builds also feed fuzzers (Part F).

TWO C/C++ TOOLCHAINS

```
# GCC toolchain: gcov + lcov
gcc --coverage prog.c -o prog && ./prog    # emits .gcda / .gcno
gcov prog.c    # line counts    lcov + genhtml → HTML report
# LLVM toolchain: source-based, region-precise
clang -fprofile-instr-generate -fcoverage-mapping prog.c ; llvm-cov show
```

SOURCE-BASED > LINE-BASED llvm-cov tracks regions (parts of a line), giving more precise branch/condition data than classic gcov line counting.

The rest — .NET · Go · Rust · Ruby · PHP

Every mainstream stack has a maintained, mostly-free coverage tool.

.NET

coverlet (standard collector) · dotnet test --collect:"XPlat Code Coverage" · Fine Code Coverage (VS) · dotCover.

Go (built-in)

go test -cover ./... · -coverprofile=cov.out · go tool cover -html.
Go 1.20+ adds integration-test coverage.

Rust

cargo-llvm-cov (recommended, region-level) · grcov (Mozilla) · cargo-tarpaulin.

Ruby / PHP

SimpleCov (drop-in) · PHPUnit with Xdebug or PCOV as the coverage driver.

JaCoCo vs Istanbul — at a glance

The two dominant coverage tools, side by side.

	JaCoCo (Java)	Istanbul / nyc (JS/TS)
Instruments	bytecode (Java agent)	source (Babel) or V8-native
Metrics	instruction, branch, line, method, class	statement, branch, function, line
Reports	HTML, XML, CSV	HTML, lcov, text, JSON
CI / thresholds	COVEREDRATIO gate; Jenkins, Sonar	check-coverage; Codecov, Sonar
License	EPL — free/OSS	ISC — free/OSS

PART E

CI & dashboards

Track coverage over time, and gate merges on the coverage of what changed.



Coveralls & Codecov

Upload a report; get trend tracking, PR comments and patch coverage.

Coveralls

Multi-language. Connect GitHub → add JaCoCo + Coveralls plugin → push report → web trend.

Per file. Per-file coverage and full build history.

Codecov

Aggregation. PR comments, patch/diff coverage & gates. Free for OSS.

2024–26. Codecov: Sentry-owned (2022) → Harness (2026).

HOW THEY FIT both plug into CI: your test job emits a coverage report (XML/lcov), the service ingests it and comments patch coverage on the PR.

SonarQube — “Clean as You Code”

Coverage + static analysis with a quality gate on NEW code.

1. **New-code gate** — default gate requires $\geq 80\%$ coverage on NEW code — legacy debt doesn't block you.
2. **More than coverage** — bugs, vulnerabilities, code smells, duplication, complexity — one gate.
3. **2024 rename** — SonarCloud → SonarQube Cloud; self-hosted → SonarQube Server (Community free).

THE IDEA gate quality on the diff you're shipping now, not the whole repo — so a large legacy codebase can improve steadily.

SonarQube — measures & risk views

Coverage plotted against complexity and technical debt tells you WHERE to test.

Line coverage per file

A ranked measures view: which files are least covered, sorted so the gaps are obvious.

Risk bubble — coverage vs debt

Bubbles plot coverage against technical debt & complexity; size = lines. Big, low bubbles are your next targets.

READ IT big bubbles low on the coverage axis = complex, poorly-tested files = your next test targets.

Gate on the diff, not the total

Enforce quality where it's cheap: the lines this PR changed.

PATCH GATE (codecov.yml)

```
# codecov.yml
coverage:
  status:
    project:
      default: { target: 80% }
    patch:
      default: { target: 90% } # diff
```

UPLOAD + LOCAL DIFF-COVER

```
# GitHub Actions
- run: pytest --cov=pkg \
  --cov-report=xml
- uses: codecov/codecov-action@v5
  with: { files: coverage.xml }
# local: diff-cover coverage.xml
```

WHY PATCH, NOT TOTAL total coverage is dominated by legacy code and barely moves. Requiring 90% on CHANGED lines raises quality steadily.

PART F

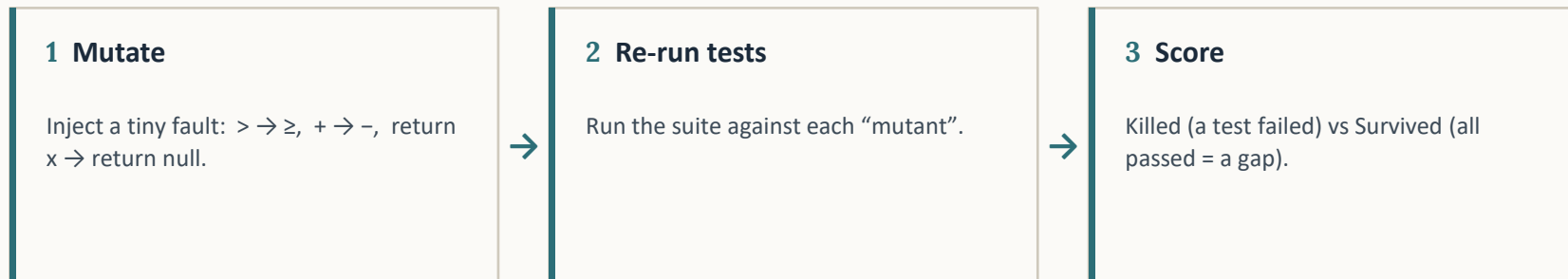
Beyond coverage & AI

Mutation testing, fuzzing, and LLM-generated tests — the modern answer to “coverage $\neq$ quality”.



Mutation testing — how it works

Inject small faults; a good test suite should notice.



MUTATION SCORE = killed ÷ total mutants it measures whether tests DETECT behavioural change — the gap coverage can’t see.

Example: `pytest --cov` reports 87%, but a `mutmut` run scores 61% — the 26-point gap is executed-but-unchecked behaviour.

The Triangle classifier — coverage vs mutation

The classic example. Five tests reach 100% statement AND branch coverage — but do they check the logic?

SUT — classify a triangle

```
String triangle(int a,int b,int c){  
    if (a<=0 || b<=0 || c<=0) return "Invalid";  
    if (a+b<=c || a+c<=b || b+c<=a)  
        return "NotATriangle";  
    if (a==b&&b==c) return "Equilateral";  
    if (a==b || b==c || a==c) return "Isosceles";  
    return "Scalene"; }
```

WEAK SUITE — 100% COVERAGE

```
triangle(2,2,2) == Equilateral  
triangle(2,2,3) == Isosceles  
triangle(3,4,5) == Scalene  
triangle(1,2,9) == NotATriangle  
triangle(0,1,1) == Invalid  
coverage.py: 100% stmt · 100% branch
```

THE TRAP five tests run every line and every branch — the coverage report says ‘done’. Mutation asks a harder question: would a bug survive?

100% coverage, 67% mutation — two survivors

Inject 6 mutants into the same code. Four die, two survive — and each survivor is a real test gap.

Mutant (one tiny edit)	Where	Verdict
M1 $a+b \leq c \rightarrow a+b < c$	triangle-inequality boundary	SURVIVED
M2 $(a==b \ \&\& \ b==c) \rightarrow $	equilateral condition	KILLED
M3 drop the 'a==c' clause	isosceles condition	SURVIVED
M4 $a \leq 0 \rightarrow a < 0$	invalid guard boundary	KILLED
M5 $a + b \rightarrow a - b$	arithmetic operator	KILLED
M6 $b==c \rightarrow b \neq c$	equilateral condition	KILLED

THE FIX add `triangle(1,2,3)=NotATriangle` + each-clause isosceles (5,4,5),(2,3,3),(3,2,2) $\rightarrow$ mutation 4/6 (67%) $\rightarrow$ 6/6 (100%). Coverage stayed 100%; only the tests changed. Runnable in Demo/triangle-demo.

Mutation testing — tools

Mature tools for every major stack — the catch is runtime cost.

1. **PIT / Pitest — Java** — mutates bytecode; scmMutationCoverage runs on changed files only.

2. **Stryker Mutator** — StrykerJS (JS/TS), Stryker.NET (C#), Stryker4s (Scala). npx stryker run.

3. **Python / Rust** — mutmut & cosmic-ray (Python) · cargo-mutants (Rust).

4. **Cost** — slow — re-runs tests per mutant → run nightly or on changed files only.

Coverage-guided fuzzing

Coverage as a search signal: keep inputs that reach new code.

How it works

the fuzzer mutates inputs and keeps those hitting NEW coverage edges — coverage becomes the fitness function.

Engines

AFL++ · libFuzzer (in-process, LLVM) · Google OSS-Fuzz (free continuous fuzzing for OSS).

Per language

Jazzer (JVM) · Atheris (Python) · cargo-fuzz (Rust) — all libFuzzer-based, OSS-Fuzz-supported.

```
clang -fsanitize=fuzzer,address prog.c -o fuzz && ./fuzz corpus/
```

Pair with sanitizers

ASan (memory) + UBSan turn silent corruption into crashes the fuzzer can find.

AI / LLMs to raise coverage (2023–2026)

Agents that generate tests and iterate until a coverage target is met.

Meta TestGen-LLM (2024)

improves existing tests behind assured filters (compiles, passes, raises coverage). 73% accepted at Meta.

Qodo Cover (open source)

agent loops generate→run→measure, keeping only tests that pass AND raise coverage.

Diffblue Cover

autonomous Java unit tests from bytecode via reinforcement learning — every test compiles & runs.

EvoSuite · Copilot · Cursor

search-based (genetic) generation; IDE “generate tests” — fast, but review before trusting.

The AI coverage trap

LLM tests can inflate coverage while checking nothing — guard against it.

The failure mode

assertion-free or tautological tests execute code and pass trivially — coverage jumps, bug-detection doesn't.

The tell

coverage 62% → 91% but PIT mutation score barely moves (58% → 60%): the new tests are padding.

Guard 1 — review

require meaningful assertions in code review; reject calls-without-checks.

Guard 2 — mutation gate

validate the AI-generated suite with mutation score, not just the coverage number.

A free student stack for coverage

Everything below is free / open-source — enough for the whole course.

Need	Tool	Cost
Java coverage	JaCoCo	Free (OSS)
JS / TS coverage	Vitest / c8 (Istanbul)	Free (OSS)
Python coverage	coverage.py + pytest-cov	Free (OSS)
Dashboards	Coveralls / Codecov (OSS)	Free tier
Mutation testing	PIT / mutmut	Free (OSS)
AI test generation	Qodo Cover	Free (OSS)

Case study — the Simple Calculator

One Java project taken through the whole toolchain.

1. **1 · JaCoCo** — measure line/branch coverage of the JUnit tests.

2. **2 · PIT (Pitest)** — mutation-test to expose weak assertions.

3. **3 · Coveralls** — publish & track coverage over builds.

4. **4 · SonarQube** — quality gate + coverage on new code.

Figure. The SUT — a console calculator taken end-to-end through coverage, mutation, dashboard and gate.

Design tests per criterion

For each predicate, give the test sets that satisfy statement, branch, condition and MC/DC.

A Two chained ifs

if (A<B) and (A>2) ... / if (B>0) and (A>0)
... — give sets for statement, branch,
condition, MC/DC.



B Loop + decision

while (x>10) x-=10; if (count<20 &&
x%2==0)... — what does path coverage
demand here?



C Predicate

$((u==0) \vee (x>5)) \wedge ((y<6) \vee (z==0))$ — build
an MC/DC set (~5 tests) and prove each
condition independent.

DELIVERABLE for one exercise, run a coverage tool + a mutation tool and report the gap between the two numbers.

REAL-WORLD STAKES • LIVE DEMO

Where an untested line is a loaded gun

In these domains an unexecuted branch is a backdoor, a fraud, or a fatality — if no test touches a line, ask who wrote it and why it can't be tested.



Lottery & betting

A payout branch no test runs is a door into the prize fund.

THE UNTESTED BRANCH

```
def payout(ticket):  
    if code == SECRET:  
        return JACKPOT  
    return normal()
```

Figure. A branch no test reaches.

Examples — Vietlott, Keno, computerised lotto, online bookmakers, betting exchanges.

Hidden branch — a payout backdoor firing only for a secret agent code; RNG interference; edited odds.

Why coverage — an unexecuted line is one nobody has confirmed — coverage is the only tool that exposes ‘runs only with a secret’.

Real case — Eddie Tipton (Multi-State Lottery, US) rigged the RNG 2005–11, ~\$2.2M defrauded; 25-year sentence, 2017.

THRESHOLD 100% branch coverage on the RNG & payout modules; every ‘agent / promo code’ branch must have a test.

Core banking & payments

On the money path, every branch must be exercised by a test.

THE UNTESTED BRANCH

```
def transfer(a, b, amt):  
    if flag: fee = 0  
    if flag: limit = INF  
    post(a, b, amt)
```

Figure. A branch no test reaches.

Examples — core banking ledgers, e-wallets, payment gateways, interbank transfer.

Hidden branch — fee-waiver / limit-raise / silent-transfer paths; a rare rounding branch that corrupts balances.

Why coverage — every path that moves money must be tested — an uncovered branch is insider-fraud or leakage risk.

Real case — Knight Capital (2012): a reused flag re-activated old dead code, never tested → \$440M loss in 45 minutes.

THRESHOLD 100% coverage on the accounting core; a diff / patch-coverage gate on every change to a money path.

Medical devices

Safety of life: no dose-related branch may be left 'never run'.

THE UNTESTED BRANCH

```
def set_dose(mode):  
    if race_cond:  
        dose *= 100  
    deliver(dose)
```

Figure. A branch no test reaches.

Examples — radiotherapy machines, ventilators, insulin pumps, infusion & dosing controllers.

Hidden branch — a rare error path or race condition that has never been tested → overdose.

Standard — IEC 62304 (medical software lifecycle), ISO 14971 (risk management); FDA expects coverage evidence.

Real case — Therac-25 (1985–87): an untested race-condition branch caused radiation overdoses; ≥ 6 incidents, ≥ 3 deaths.

MANDATORY branch / condition coverage on every dose-related path — no branch may remain unexecuted.

Automotive / aviation / rail

Compound safety conditions must be covered to MC/DC.

THE UNTESTED BRANCH

```
def brake(sensor):  
    if fault and low:  
        skip_ABS()  
    apply()
```

Figure. A branch no test reaches.

Examples — ABS/ESP brake ECUs, cockpit software, railway signalling & interlocking.

Hidden branch — a compound condition (fault AND low-speed) left uncovered → loss of a safety function.

Standard — DO-178C Level A (avionics) & ISO 26262 ASIL-D (automotive) mandate coverage up to MC/DC.

Real case — Ariane 5 Flight 501 (1996): an unguarded, untested overflow branch destroyed the rocket; ~\$370M lost.

MC/DC each sub-condition must independently affect the outcome — about $n+1$ tests for n conditions.

Electronic voting

A rare branch is exactly where vote-rigging hides.

THE UNTESTED BRANCH

```
def count(vote):  
    if cand == A:  
        tally += 2  
    tally += 1
```

Figure. A branch no test reaches.

Examples — optical-scan counters, DRE machines, e-voting tally & aggregation systems.

Hidden branch — fraud logic in a rare condition that reweights or miscounts ballots.

Why coverage — an untested branch is one nobody has proven absent; code audit + coverage are the guard.

Real case — Hursti Hack (2005) on Diebold AccuVote altered results via a memory card; DEF CON Voting Village broke every machine surveyed.

PRINCIPLE tally code open + 100% branch coverage + paper reconciliation (VVPAT) as an independent check.

Smart contracts / DeFi

Deployed code is immutable — cover 100% before it ships.

THE UNTESTED BRANCH

```
function draw():  
  if x == owner:  
    sendAll(x)  
    transfer(amt)
```

Figure. A branch no test reaches.

Examples — DEX exchanges, cross-chain bridges, lending & staking pools.

Hidden branch — a hidden-drain path, reentrancy, or a missing access-control check.

Why coverage — you cannot patch after deploy — every branch must be covered and fuzzed before mainnet.

Real cases — The DAO (2016) reentrancy ~\$60M; Ronin Bridge (2022) ~\$625M; Poly Network (2021) ~\$611M.

STANDARD 100% branch coverage + property/invariant tests + fuzzing (Echidna / Foundry) before mainnet.

Crypto / security libraries

The rare error branch is where keys leak and auth is bypassed.

THE UNTESTED BRANCH

```
def verify(sig):  
    try: check(sig)  
    except: return True  
    return ok
```

Figure. A branch no test reaches.

Examples — TLS/SSL stacks, signature verification, KMS/HSM, cryptographic primitives.

Hidden branch — a rarely-taken error path skipped in testing → auth bypass or key disclosure.

Standard — CWE-347 (improper signature verification); the failure path of every security check must be covered.

Real cases — Apple ‘goto fail’ (CVE-2014-1266) skipped TLS cert checks; Heartbleed (CVE-2014-0160) leaked memory on ~17% of HTTPS servers.

MANDATORY 100% coverage of error-handling branches — test the ‘failure’ path of every security check.

Tax / social-insurance / payroll

A wrong rare branch is wrong across thousands of records.

THE UNTESTED BRANCH

```
def tax(income):  
    if income < 0:  
        return 0  
    return rate*income
```

Figure. A branch no test reaches.

Examples — income-tax / VAT engines, social-insurance systems, payroll software.

Hidden branch — rare-case formulas (negative income, holidays, back-pay) that miscalculate or are tampered with.

Why coverage — one wrong branch corrupts records en masse; an untested branch easily hides fraud logic.

Real case — Post Office Horizon (UK, 2000–14): faulty branches produced phantom shortfalls → 900+ prosecutions, ~700 wrongful convictions; >£1bn redress.

THRESHOLD 100% branch coverage on the calculation engine; parallel-run reconciliation on every formula change.

Vietlott payout — 10 tests pass, 81% covered

A real, reproducible coverage.py run. SUT: calculate_payout() for a 6/45 ticket.

THREE COMMANDS, ONE REAL RUN

```
# Python 3.8+
pip install -r requirements.txt
coverage run -m pytest -q
..... [100%] 10 passed
coverage report -m payout.py
coverage html # red lines, visually
```

SUT — calculate_payout(ticket) prices one Vietlott 6/45 ticket.

Suite — 10 unit tests, all green (missing numbers, wrong count, each tier).

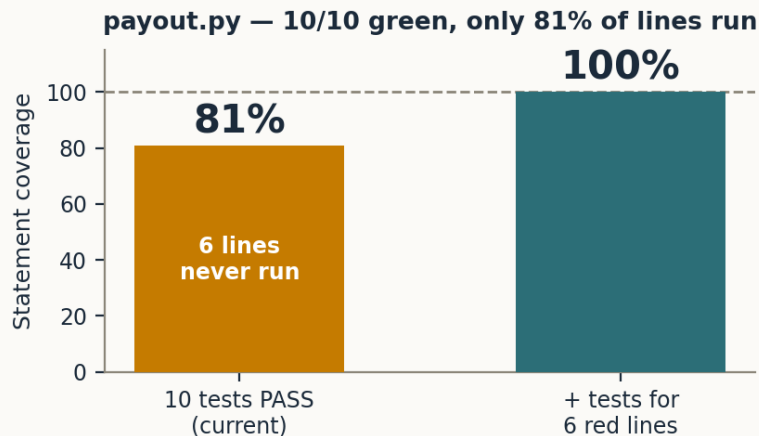
Result — 10/10 pass, yet only 81% of statements execute; 6 lines never run.

Attached — the runnable project ships in Demo/demo-code-coverage.

MEASURED Python 3.10 — 81% statement / 83% branch. A measured number, not an estimate.

The blind spot — 10 green tests, 6 lines never run

Every test passes, yet 19% of statements never execute. coverage.py names exactly which lines.



```
$ coverage report -m payout.py
payout.py 31 stmts 6 miss 81%
Missing: 26, 31-32, 49, 53-54
```

L26

Hidden backdoor — secret agent code

L31-32

Defensive, untested — bad-draw except

L49

Missing test — SMS 5% fee

L53-54

Dead code — if matched>6 impossible

CLASSIFICATION one backdoor · one defensive · one missing test · one dead-code line — only coverage surfaced them.

What the red lines mean

Four reasons a line stays untested — same 81%, but each demands a different action.

Line(s)	Type	Why it never ran	Action
26	Backdoor	fires only on a secret agent_code	security report + remove
31–32	Defensive	except never triggered (all draws valid)	add a bad-input test
49	Missing test	the SMS 5%-fee path is untested	add a channel='SMS' test
53–54	Dead code	if matched>6 is impossible (max 6)	delete it

THE WORKFLOW run coverage → inspect each red line → classify → add a test, delete, or report. With AI: generate tests for red lines, but audit — AI may ‘satisfy’ a backdoor silently.

The attached folder — run it yourself

Everything needed to reproduce the 81% number ships in Demo/demo-code-coverage.

WHAT'S IN THE FOLDER

```
demo-code-coverage/  
payout.py      # SUT  
test_payout.py # 10 tests, all pass  
requirements.txt # pytest, coverage  
run.sh / run.bat # one-command run  
README.md      # walkthrough + tasks
```

Reach 100% — write tests for the 6 red lines; you'll trip over the backdoor at line 26.

Classify — missing test, dead code, or a backdoor to report.

Stmt vs branch — re-run with --branch; show 100% statement but not 100% branch → why MC/DC.

AI audit — ask an AI to cover the red lines, then check: did it flag line 26?

PATH Mutation Testing/Demo/demo-code-coverage/ · measured: 81% statement, 83% branch (Python 3.10).

Key takeaways

Four things to remember about coverage.

Coverage is a floor, not a ceiling

it finds untested code; it never proves behaviour is checked.

Match criterion to risk

branch for everyday code; MC/DC for safety-critical (DO-178C, ISO 26262).

Gate on the diff

patch coverage + a quality gate on new code beats chasing a global %.

Verify with mutation

mutation score catches the gaps — especially for AI-generated tests.